



Natural Language Processing: BERT, GPT

HSE Faculty of Computer Science

Murat Khazhgeriev



Table of Content

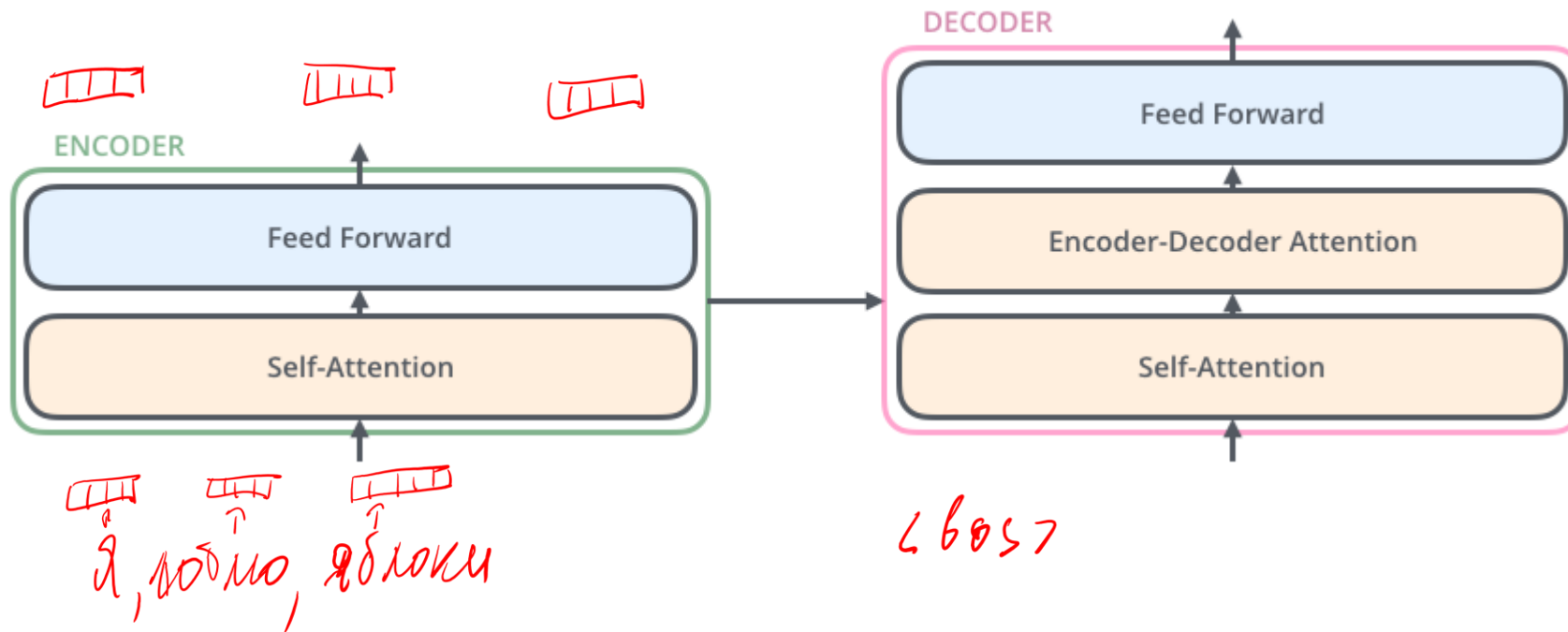
- Recap
- BERT
- GPT
- How to (pre)train your dragon transformer?
- BERT's Eleven



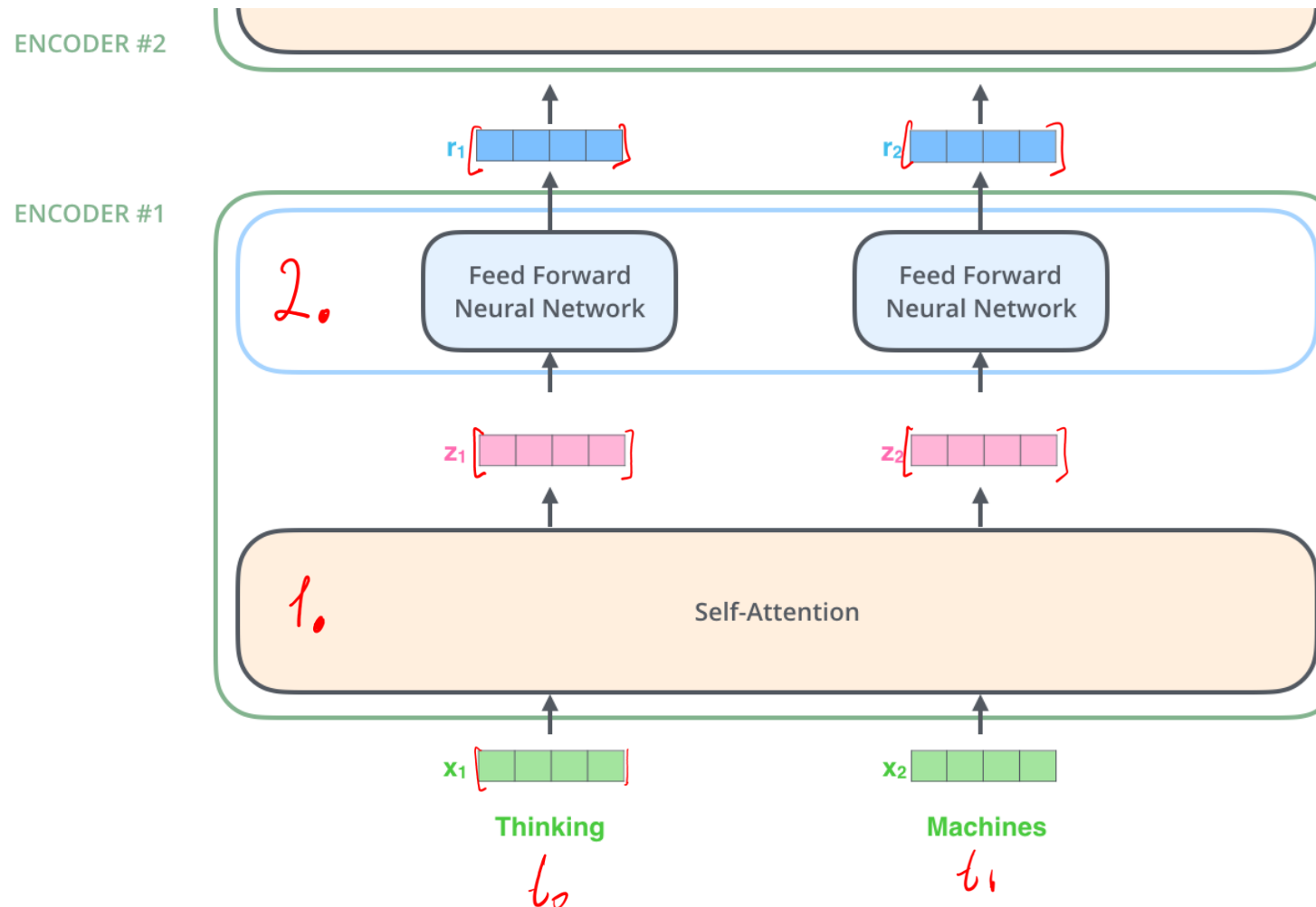
Table of Content

- **Recap**
- BERT
- GPT
- How to (pre)train your dragon transformer?
- BERT's Eleven

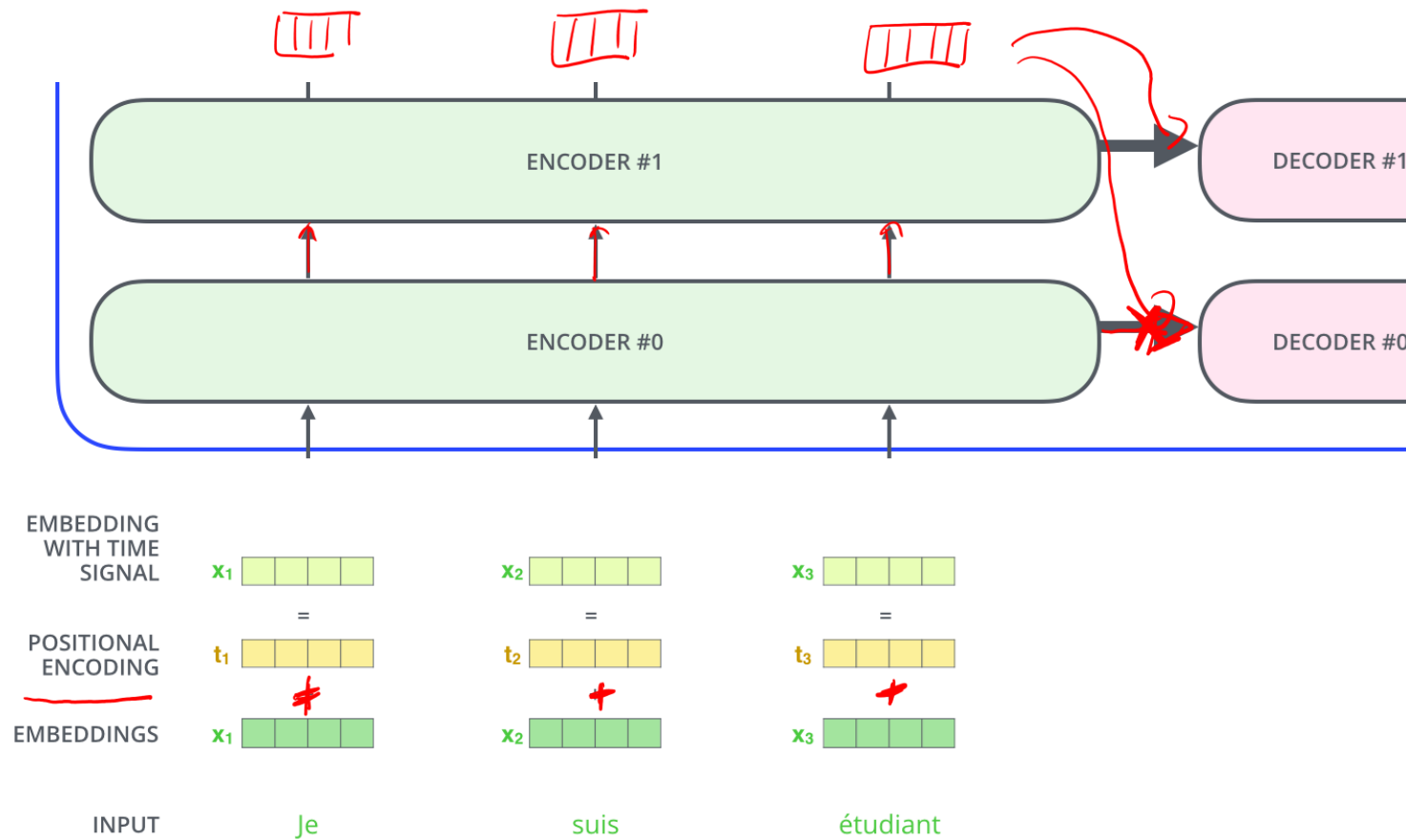
Always two there are. No more, no less. An Attention and an FFN. © Yoda
ruskua: I love apples



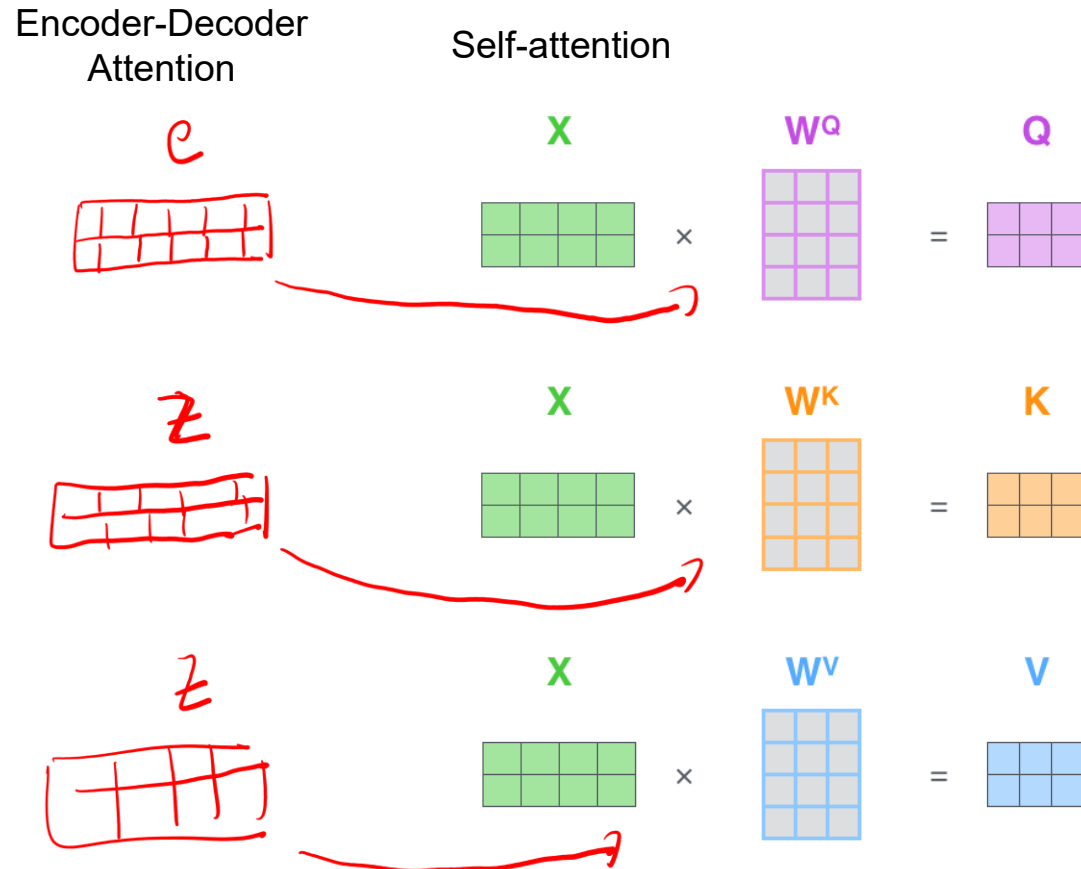
Always two there are. No more, no less. An Attention and an FFN. © Yoda



As opposed to RNNs, Transformers do not track the position implicitly



Inside a Self-Attention: Matrix View



Inside a Self-Attention: Matrix View

$$\text{softmax} \left(\frac{\overset{\text{Q}}{\overset{\text{1} \quad \text{2}}{\begin{matrix} \square & \square & \square \\ \square & \square & \square \end{matrix}}} \times \overset{\text{K}^T}{\begin{matrix} \square & \square \\ \square & \square \\ \square & \square \end{matrix}}}{\sqrt{d_k}} \right) \overset{\text{V}}{\begin{matrix} \square & \square & \square \\ \square & \square & \square \end{matrix}}$$

$$= \overset{\text{Z}}{\begin{matrix} \square & \square & \square \\ \square & \square & \square \end{matrix}}$$

$$QK^T = \begin{bmatrix} \langle q_1, k_1 \rangle & \langle q_1, k_2 \rangle \\ \langle q_2, k_1 \rangle & \langle q_2, k_2 \rangle \end{bmatrix}$$

Attention is all you need

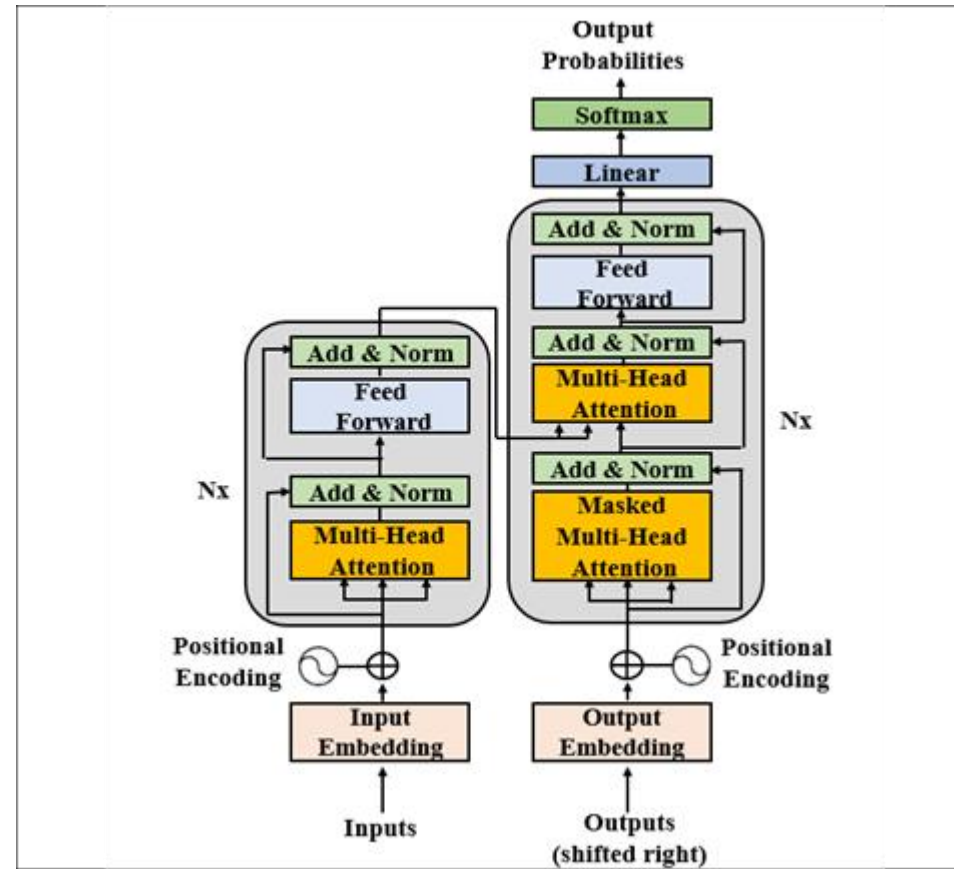
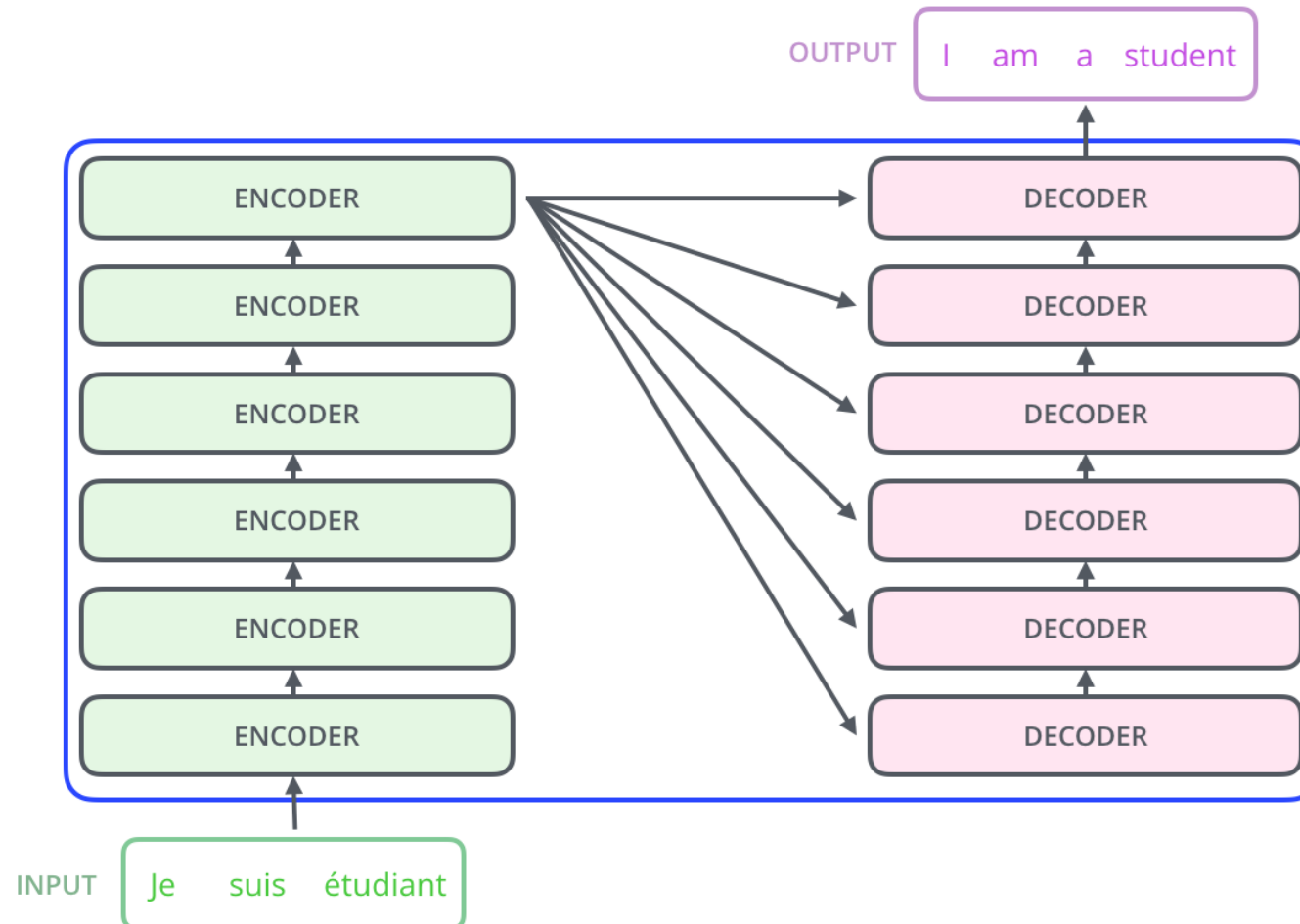




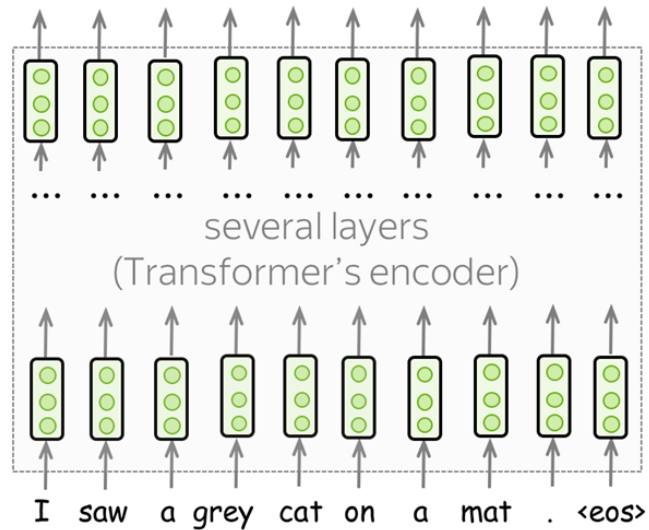
Table of Content

- Recap
- **BERT**
- GPT
- How to (pre)train your ~~dragon~~ transformer?
- BERT's Eleven

BERT is just an Encoder part



Using encoder as an embedding generator



Model architecture:

- Transformer's encoder

What is special about it:

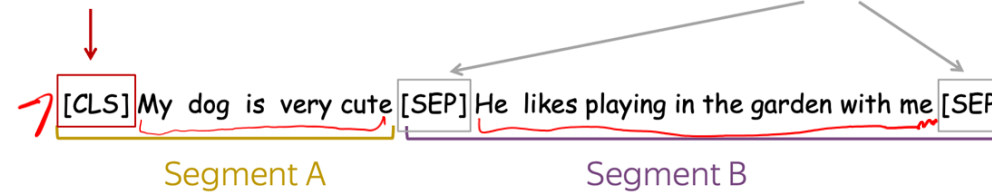
- Training objectives
 - MLM: Masked language modeling
 - NSP: Next sentence prediction
- The way it is used
 - No task-specific models

Objective one: tell whether the two sequences are consecutive

[CLS]: Special token

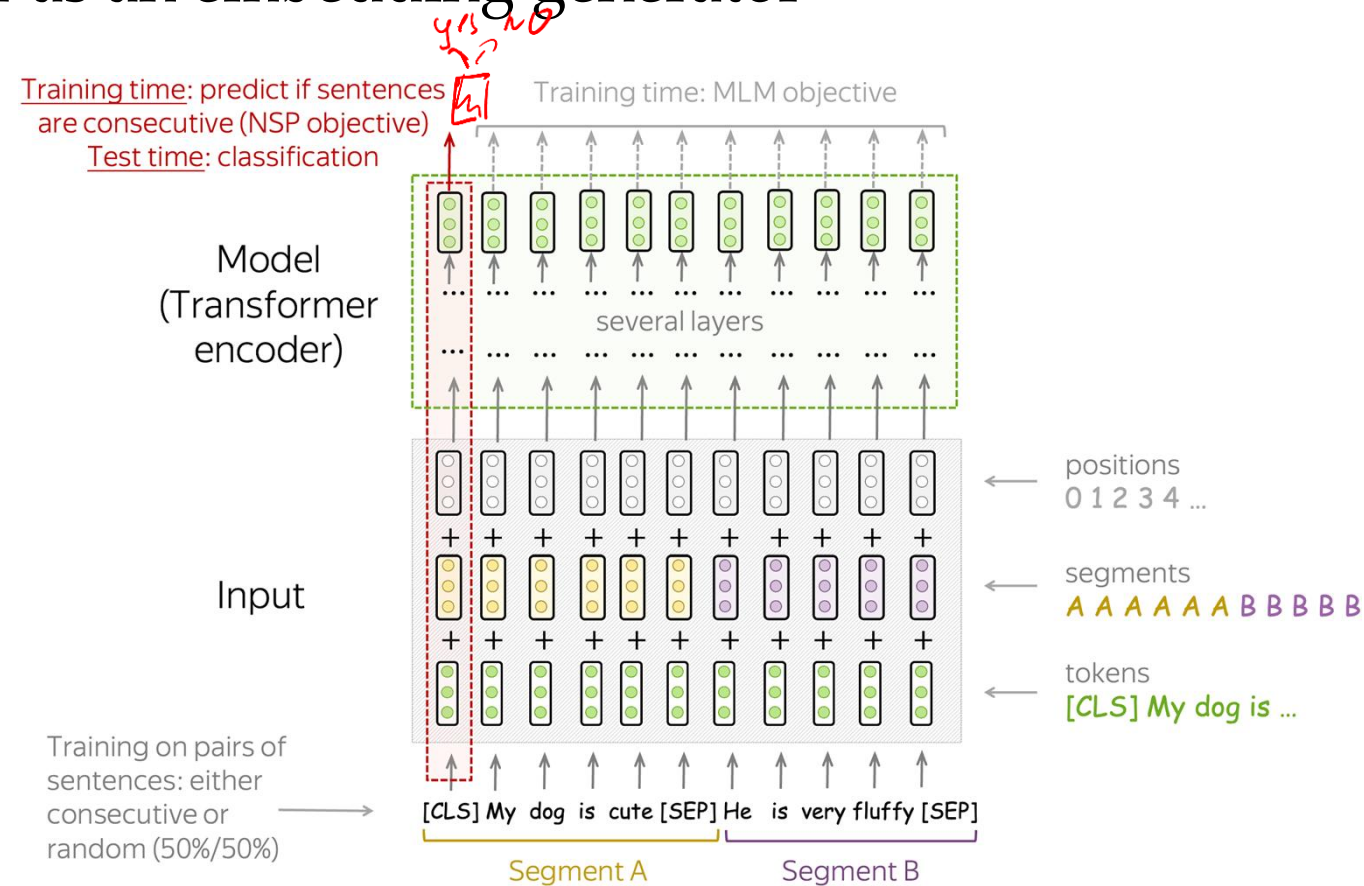
- Training time: predict if sentences are consecutive or not (Next Sentence Prediction /NSP objective)
- Test time: downstream tasks (e.g., classification)

[SEP]: Special token-separator

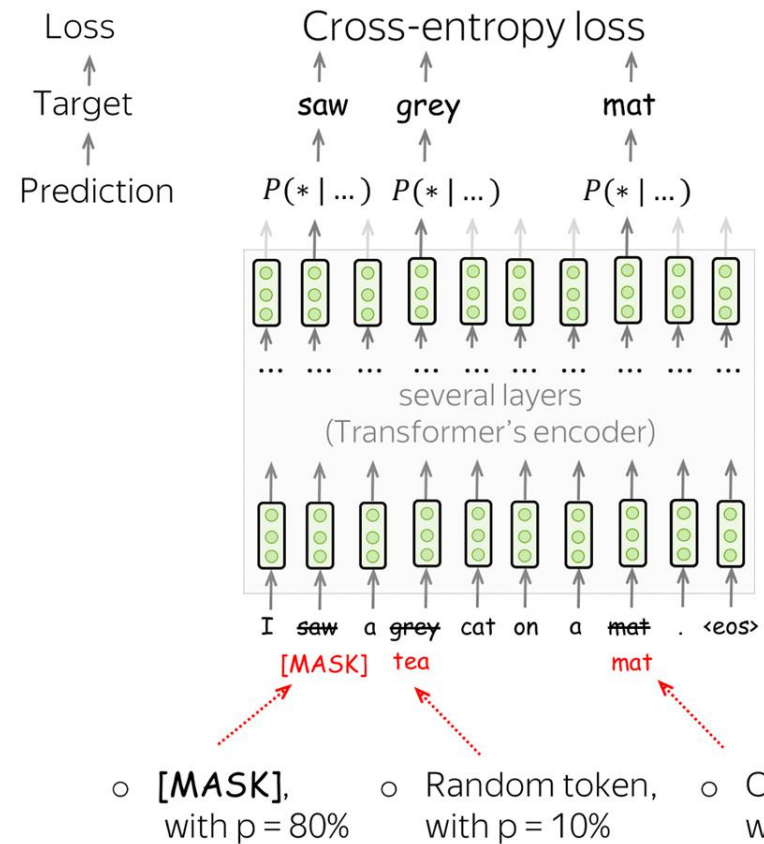


Training on pairs of sentences: either consecutive or random (50%/50%)

Using encoder as an embedding generator



Objective two: Masked Language Modeling



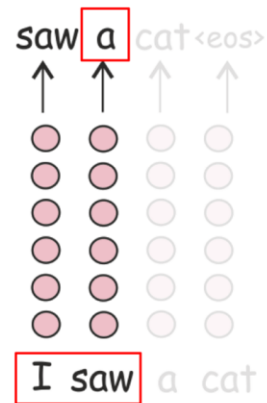
At each training step:

- pick randomly 15% of tokens
- replace each of the chosen tokens with something
- predict original chosen tokens

LM vs. MLM

Language Modeling

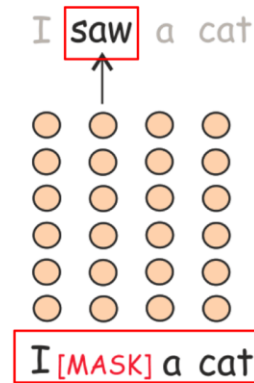
- Target: next token
- Prediction: $P(* | \text{I saw})$



left-to-right, does
not see future

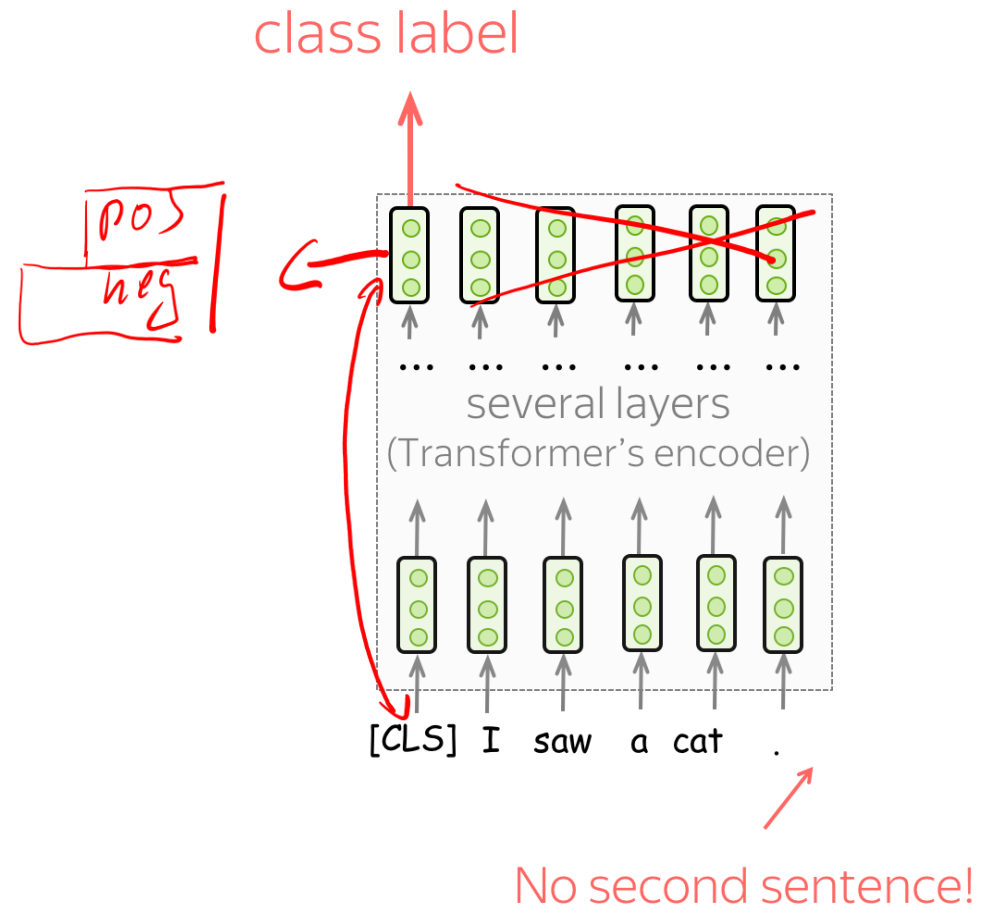
Masked Language Modeling

- Target: current token (the true one)
- Prediction: $P(* | \text{I [MASK] a cat})$

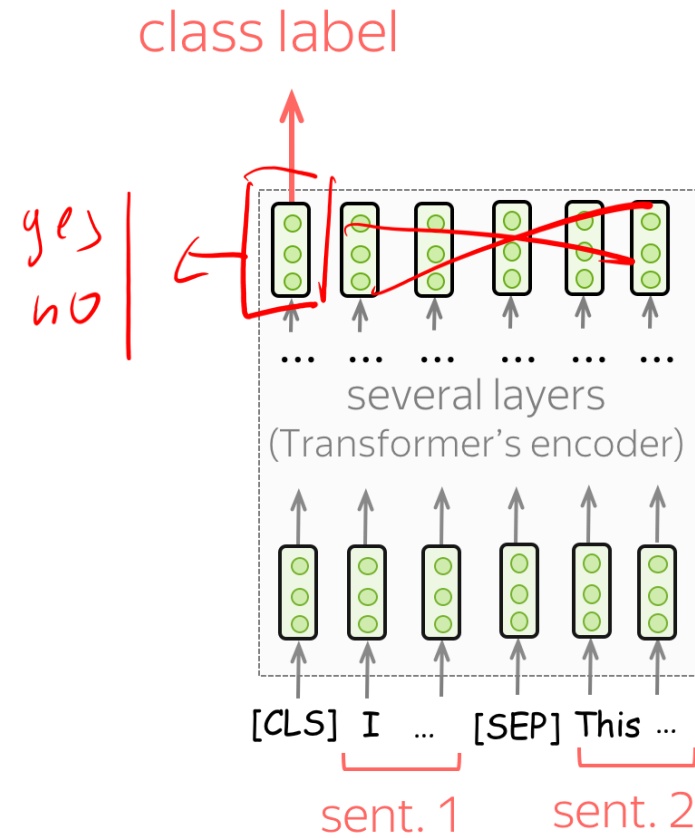


sees the whole text, but
something is corrupted

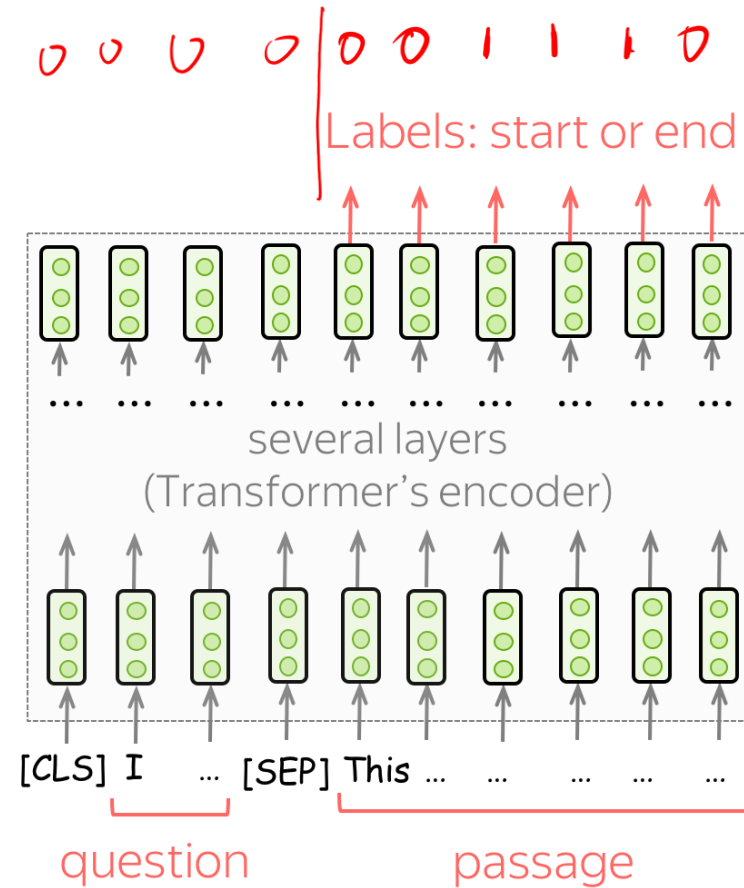
Single Sentence Classification



Sentence Pair Classification



Question Answering

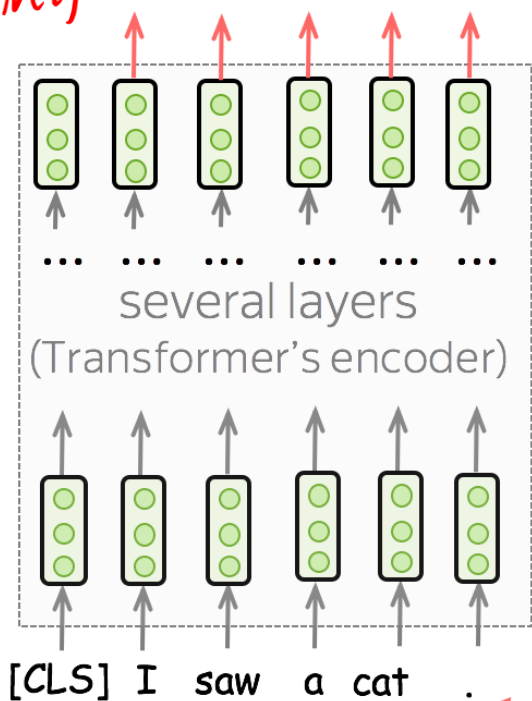


Input tagging

Named Entity Recognition (NER)

target: (no name), (very low) neo

Labels: tags



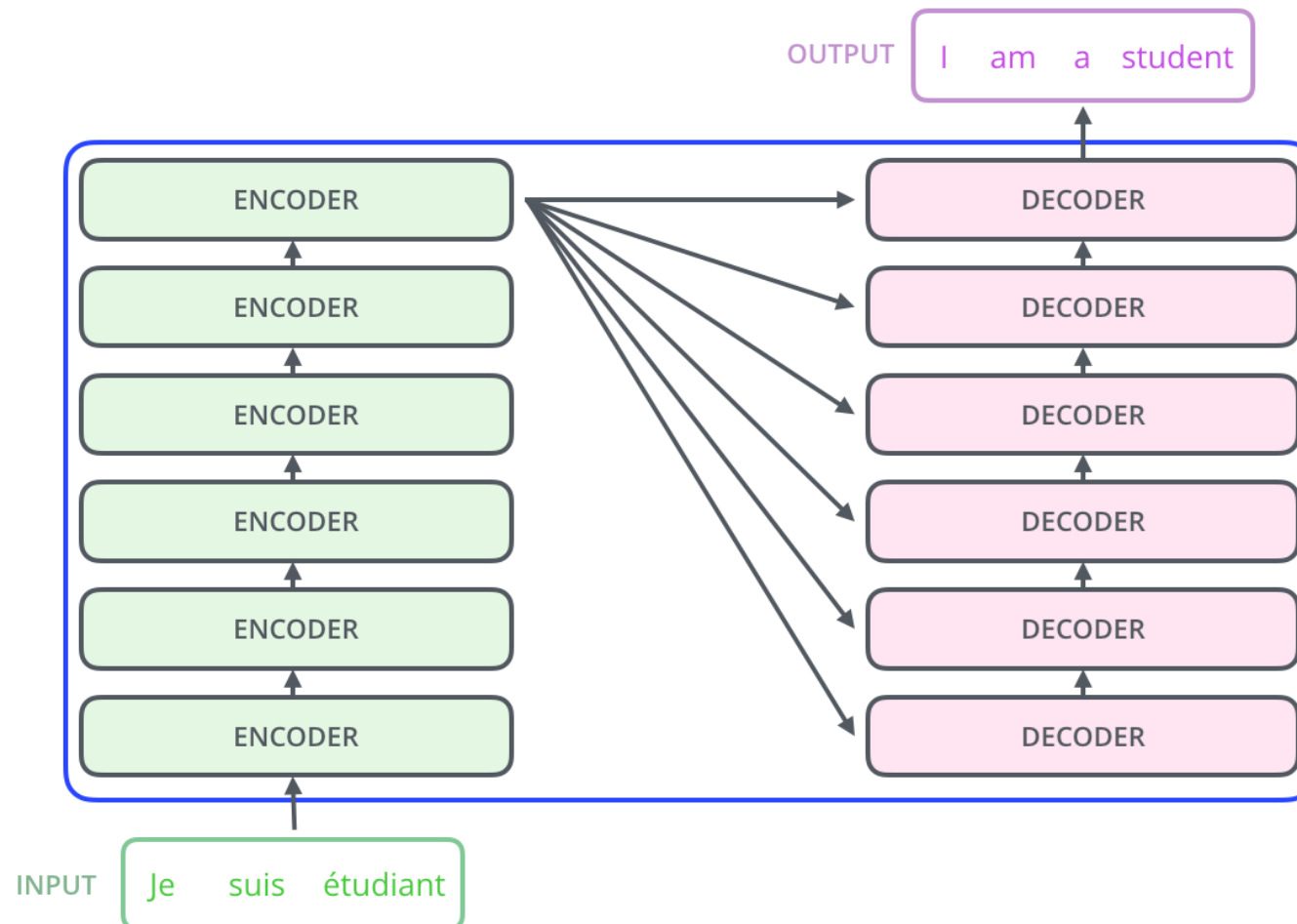
No second sentence



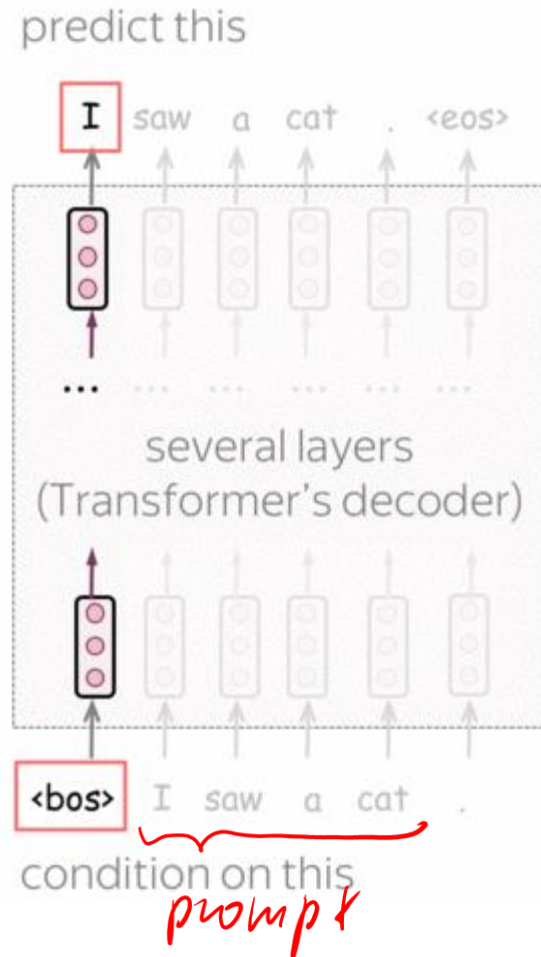
Table of Content

- Recap
- BERT
- **GPT**
- How to (pre)train your ~~dragon~~ transformer?
- BERT's Eleven

GPT is just a Decoder part



Decoder as a universal model



Decoder as a universal model

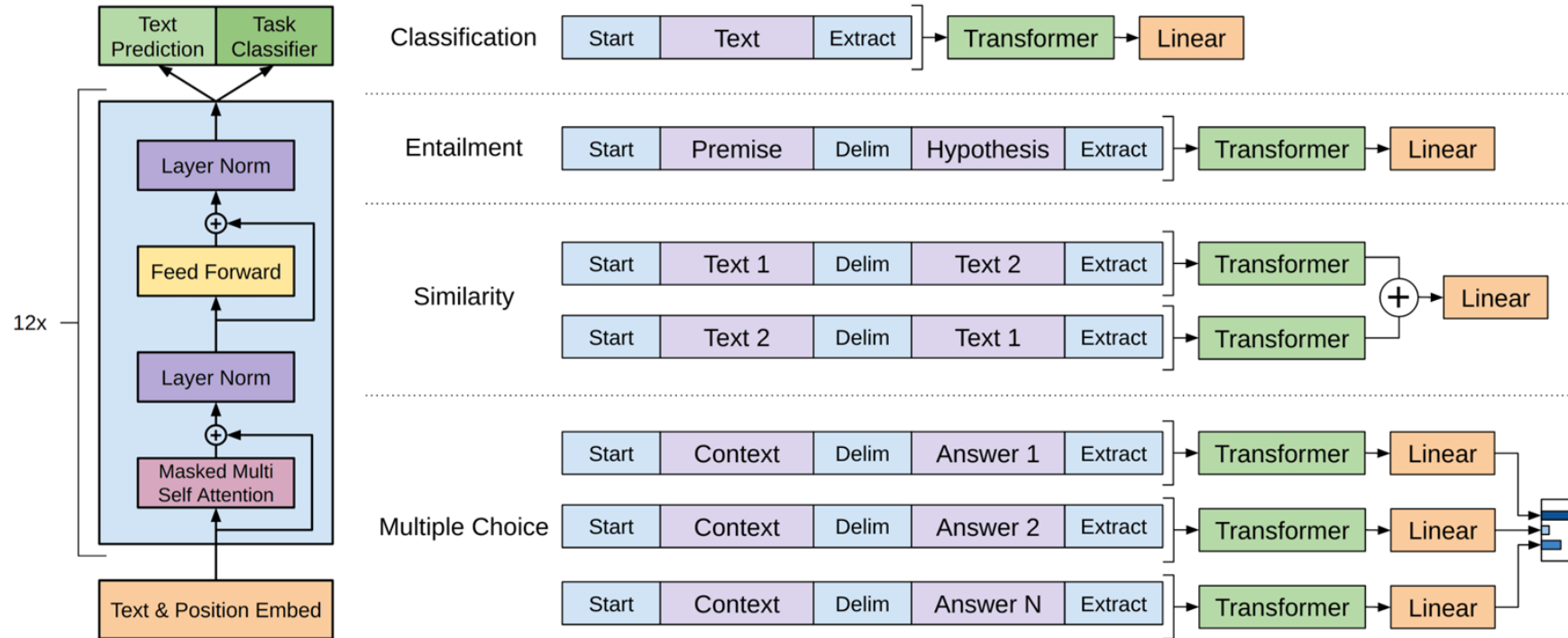


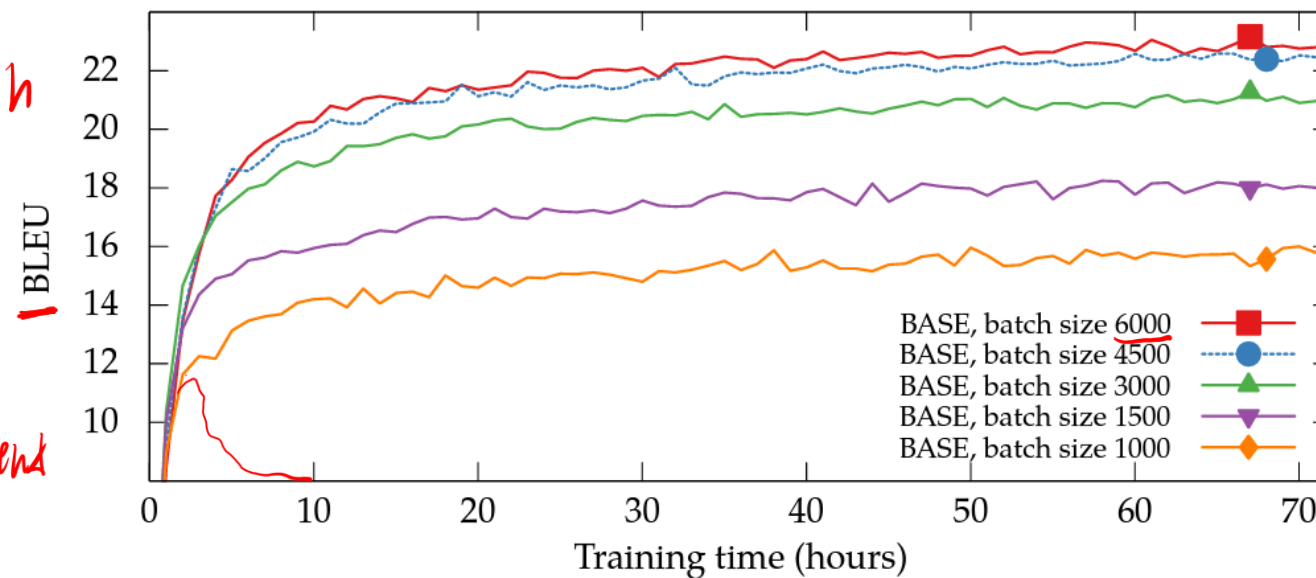


Table of Content

- Recap
- BERT
- GPT
- **How to (pre)train your ~~dragon~~ transformer?**
- BERT's Eleven

Batch size is crucial for the training

Gradient
Accumulation
 N_x { 0. Forward
1. loss
2. Gradient descent



$t_0, t_1, t_2 \#1$
 $t'_0, t'_1, t'_2 \#2$

Figure 5: Effect of the batch size with the BASE model. All trained on a single GPU.

Sequence max length is also important

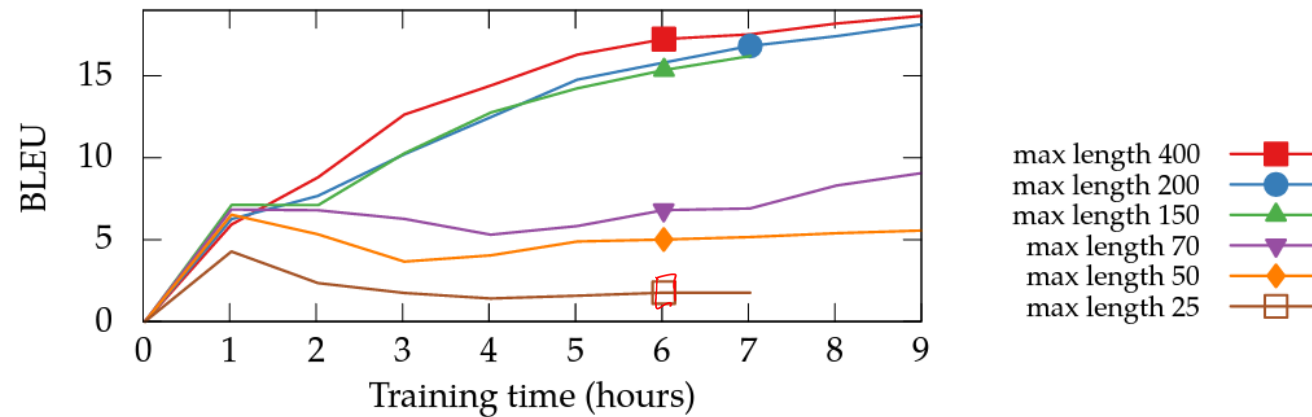
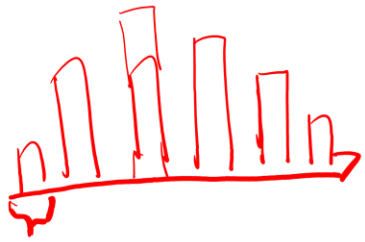
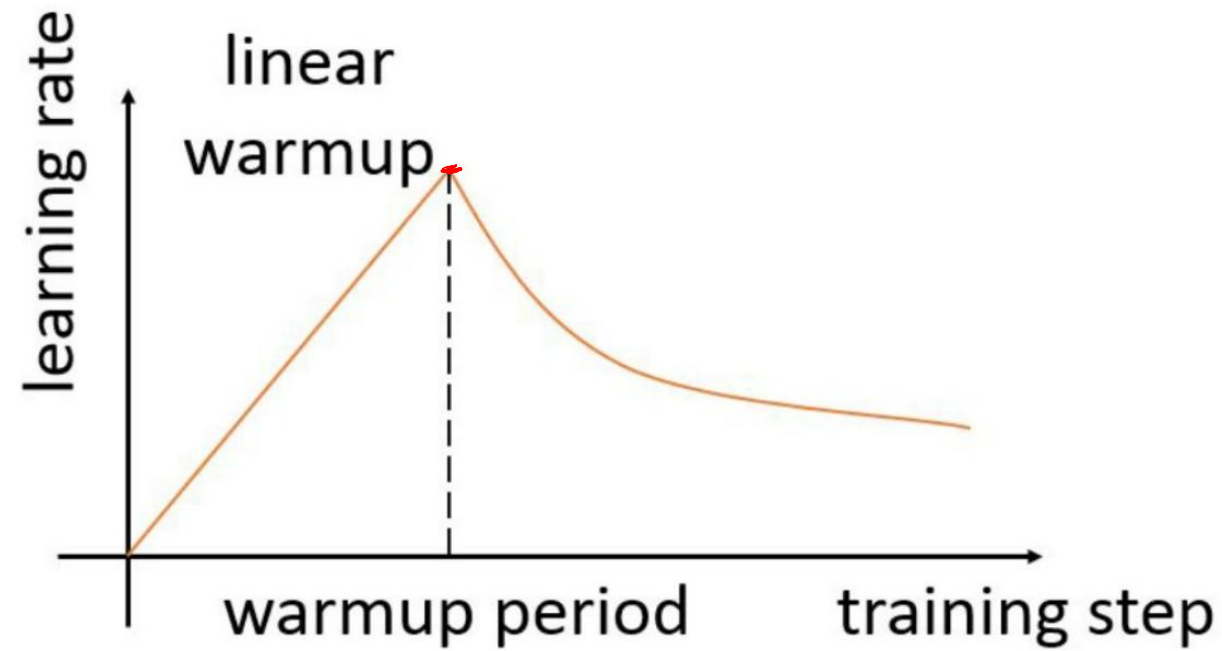


Figure 4: Effect of restricting the training data to various max_length values. All trained on a single GPU with the BIG model and batch_size=1500. An experiment without any max_length is not shown, but it has the same curve as max_length=400.

Handwritten notes in red:

- Close key! pad pads
- Close to d1 to...
- seq-len

As is the warmup



As is the warmup

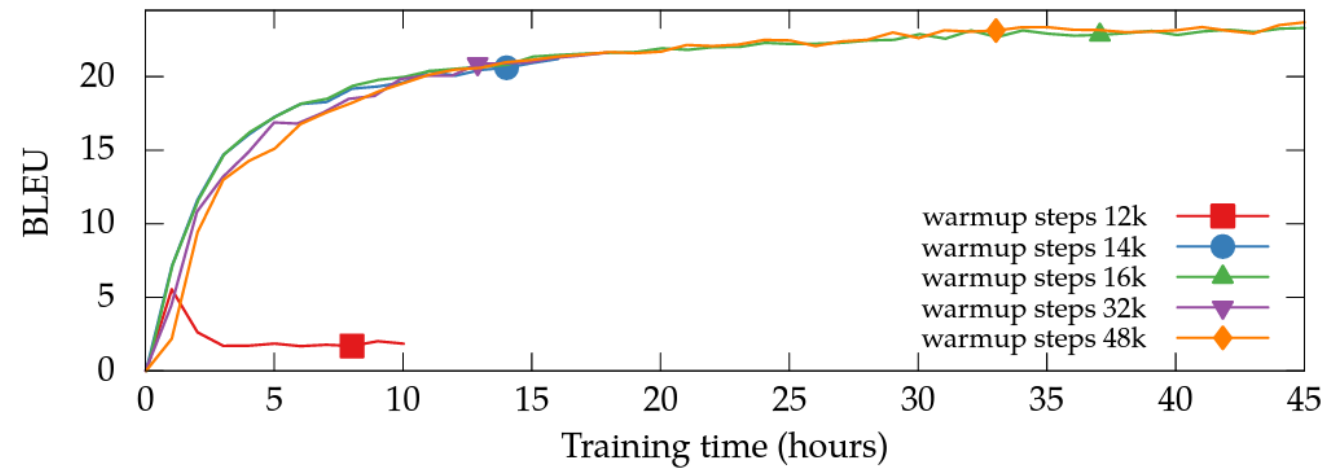


Figure 8: Effect of the warmup steps on a single GPU. All trained on CzEng 1.0 with the default batch size (1500) and learning rate (0.20).

Adam(W) from now on

input : $\gamma(\text{lr})$, $\beta_1, \beta_2(\text{betas})$, $\theta_0(\text{params})$, $f(\theta)(\text{objective})$, ϵ (epsilon)
 $\lambda(\text{weight decay})$, *amsgrad*, *maximize*

initialize : $m_0 \leftarrow 0$ (first moment), $v_0 \leftarrow 0$ (second moment), $\widehat{v}_0^{max} \leftarrow 0$

for $t = 1$ **to** $\dots$ **do**

if *maximize* :

$g_t \leftarrow -\nabla_{\theta} f_t(\theta_{t-1})$

else

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$

$\theta_t \leftarrow \theta_{t-1} - \gamma \lambda \theta_{t-1}$

$m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$

$v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$

$\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$

$\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$

if *amsgrad*

$\widehat{v}_t^{max} \leftarrow \max(\widehat{v}_t^{max}, \widehat{v}_t)$

$\theta_t \leftarrow \theta_t - \gamma \widehat{m}_t / (\sqrt{\widehat{v}_t^{max}} + \epsilon)$

else

$\theta_t \leftarrow \theta_t - \gamma \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$

return θ_t



Table of Content

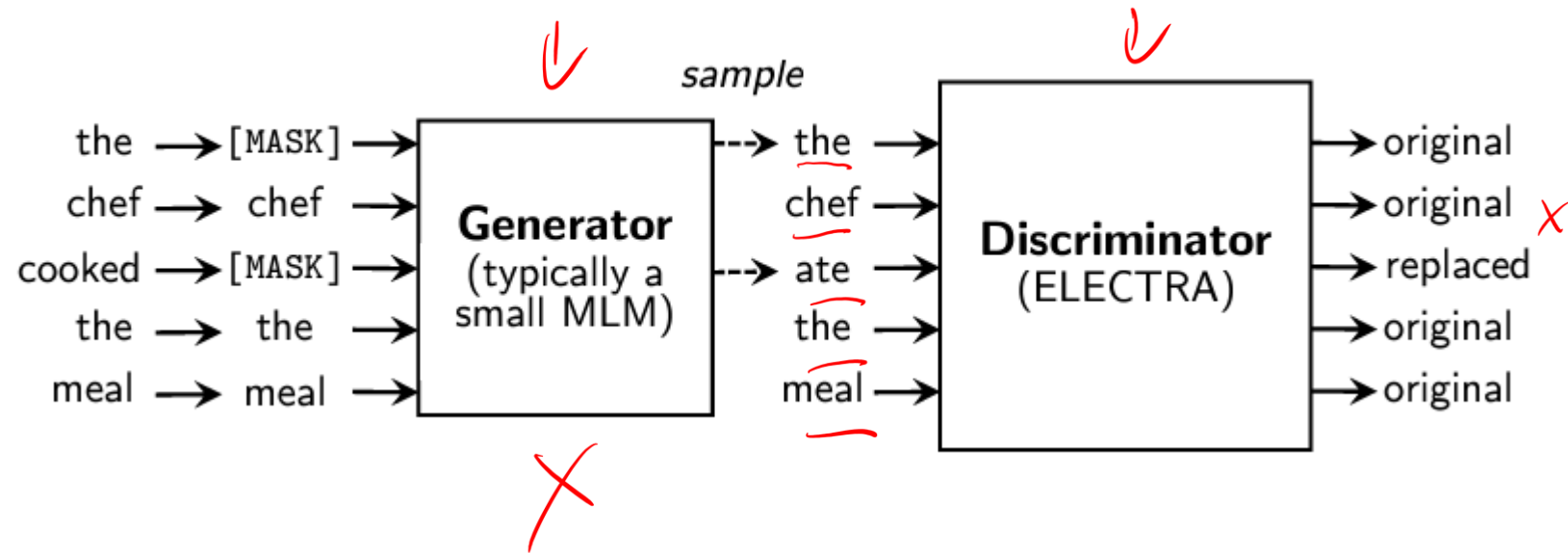
- Recap
- BERT
- GPT
- How to (pre)train your ~~dragon~~ transformer?
- **BERT's Eleven**

Vanilla BERT was badly trained (even though was SOTA on release)

Model	data	bsz	steps	SQuAD (v1.1/2.0)	MNLI-m	SST-2
RoBERTa						
with BOOKS + WIKI	16GB	8K	100K	93.6/87.3	89.0	95.3
+ additional data (§3.2)	160GB	8K	100K	94.0/87.7	89.3	95.6
+ pretrain longer	160GB	8K	300K	94.4/88.7	90.0	96.1
+ pretrain even longer	160GB	8K	500K	94.6/89.4	90.2	96.4
BERT_{LARGE}						
with BOOKS + WIKI	13GB	256	1M	90.9/81.8	86.6	93.7
XLNet_{LARGE}						
with BOOKS + WIKI	13GB	256	1M	94.0/87.8	88.4	94.4
+ additional data	126GB	2K	500K	94.5/88.8	89.8	95.6

Table 4: Development set results for RoBERTa as we pretrain over more data (16GB $\rightarrow$ 160GB of text) and pretrain for longer (100K $\rightarrow$ 300K $\rightarrow$ 500K steps). Each row accumulates improvements from the rows above. RoBERTa matches the architecture and training objective of BERT_{LARGE}. Results for BERT_{LARGE} and XLNet_{LARGE} are from Devlin et al. (2019) and Yang et al. (2019), respectively. Complete results on all GLUE tasks can be found in the Appendix.

ELECTRA introduces a supervision by another model



Make Encoders Cool Again (MEGA)

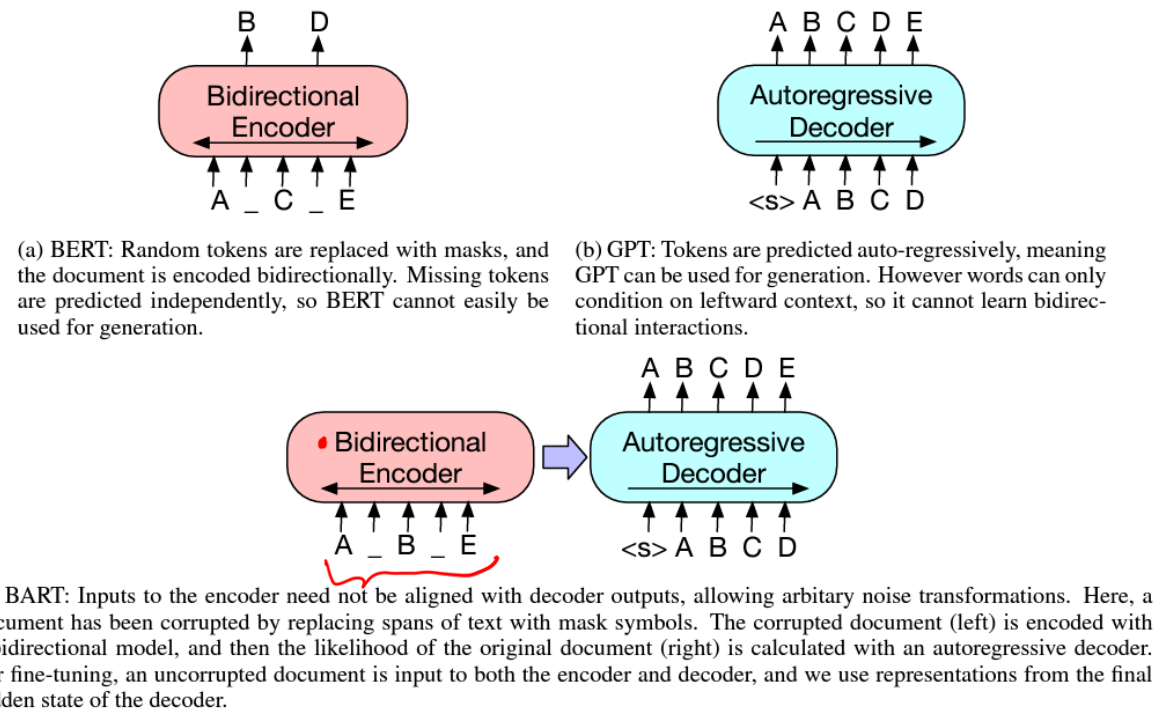


Figure 1: A schematic comparison of BART with BERT (Devlin et al., 2019) and GPT (Radford et al., 2018).



T5 is BART on Steroids

Model	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ Baseline	83.28	19.24	80.88	71.36	26.98	39.82	27.65
Baseline-1T	84.80	19.62	83.01	73.90	27.46	40.30	28.34
T5-Base	85.97	20.90	85.44	75.64	28.37	41.37	28.98

Table 15: Performance comparison of T5-Base to our baseline experimental setup used in the rest of the paper. Results are reported on the validation set. “Baseline-1T” refers to the performance achieved by pre-training the baseline model on 1 trillion tokens (the same number used for the T5 model variants) instead of $2^{35} \approx 34\text{B}$ tokens (as was used for the baseline).

T5 is BART on Steroids

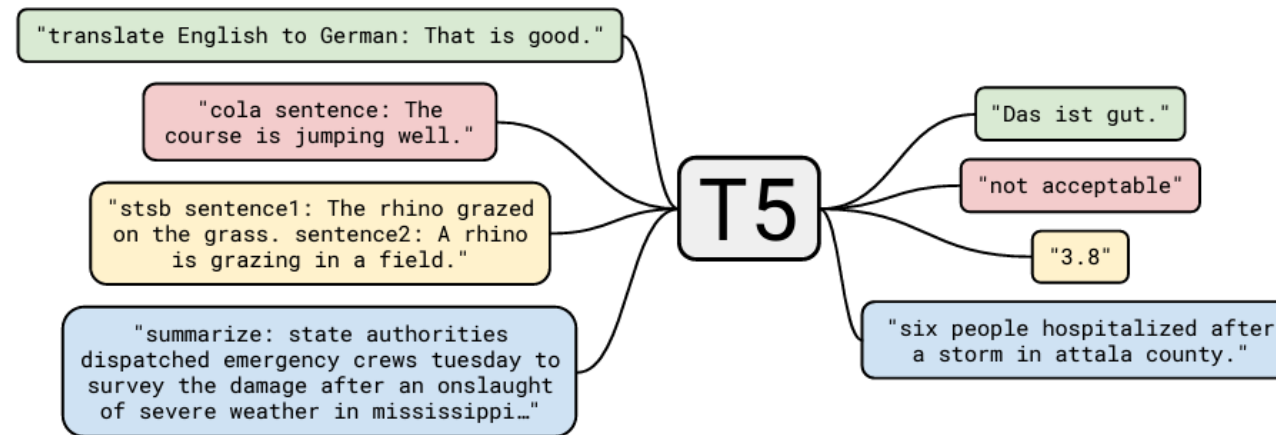


Figure 1: A diagram of our text-to-text framework. Every task we consider—including translation, question answering, and classification—is cast as feeding our model text as input and training it to generate some target text. This allows us to use the same model, loss function, hyperparameters, etc. across our diverse set of tasks. It also provides a standard testbed for the methods included in our empirical survey. “T5” refers to our model, which we dub the “**T**ext-**t**o-**T**ext **T**ransformer”.



DeBERTa is ELECTRA on steroids

Table 3: Comparison results on the GLUE development set.

Model	CoLA	QQP	MNLI-m/mm	SST-2	STS-B	QNLI	RTE	MRPC	Avg.
#Train	Mcc 8.5k	Acc 364k	Acc 393k	Acc 67k	Corr 7k	Acc 108k	Acc 2.5k	Acc 3.7k	
BERT _{large}	60.6	91.3	86.6/-	93.2	90.0	92.3	70.4	88.0	84.05
RoBERTa _{large}	68.0	92.2	90.2/90.2	96.4	92.4	93.9	86.6	90.9	88.82
XLNet _{large}	69.0	92.3	90.8/90.8	97.0	92.5	94.9	85.9	90.8	89.15
ELECTRA _{large}	69.1	92.4	90.9/-	96.9	92.6	95.0	88.0	90.8	89.46
DeBERTa _{large}	70.5	92.3	91.1/91.1	96.8	92.8	95.3	88.3	91.9	90.00
DeBERTaV3 _{large}	75.3	93.0	91.8/91.9	96.9	93.0	96.0	92.7	92.2	91.37

